

Windows 10/11

USB / Type-C

DFIR

Инженерная эксплуатация

UsbForensicAudit

Архитектура, принцип работы и интерпретация результатов

Подробное руководство для инженера: что именно сканирует приложение, зачем нужен каждый источник, как выполняется сопоставление и где заканчивается достоверный вывод.

Назначение: эксплуатация, технический аудит и проверка истории USB/Type-C устройств на работающей Windows.

Состояние проекта: 11 июля 2026 года; руководство сверено с текущей архитектурой приложения.

Уровень: Senior Engineer / DFIR-aware system engineer.

Принцип: ни один отдельный артефакт (след или запись) не даёт полной картины. Вывод строится по нескольким независимым источникам.

Как пользоваться этим руководством

Документ отвечает не на вопрос «в каком классе написан метод», а на практические вопросы инженера: **что ищется, почему это ищется, как записи связываются между собой, насколько им можно доверять и что нельзя утверждать по результату.**

Главное правило интерпретации. Наличие следа обычно доказывает, что Windows или пользовательская оболочка когда-то видели объект. Оно не всегда доказывает физическое подключение в конкретную секунду, копирование файла или умышленную очистку. Приложение специально разделяет прямые, подтверждающие и косвенные доказательства.

1. Назначение и границы
2. Архитектура простыми словами
3. Полный конвейер сканирования
4. Что проверяет каждый функциональный блок
5. Как выполняется сопоставление
6. Как читать даты, силу доказательства и уверенность
7. Как ищутся признаки очистки
8. Практический порядок работы инженера
9. Хранение, отчёты и проверяемость
10. Ограничения и запрещённые выводы
11. Каталог проверенных источников

1. Назначение и границы приложения

UsbForensicAudit — локальная программа с графическим интерфейсом (GUI) для Windows 10/11. Она собирает сохранившиеся в системе сведения об устройствах USB/Type-C, текущих подключениях, установке драйверов, томах, действиях пользователя и возможных признаках удаления следов. Затем программа приводит сведения к общей форме, объединяет записи одного физического устройства, строит временную шкалу и формирует инженерные отчёты.

Какие вопросы приложение помогает закрыть

- Какие USB-устройства Windows знает сейчас или помнит исторически?
- Каковы VID/PID, серийный номер, модель, драйвер, ContainerID и топологическое расположение?
- Какие записи относятся к одному физическому устройству, хотя Windows показывает несколько интерфейсов?

- Какой том и буква диска могли быть связаны с накопителем?
- Есть ли подтверждённые даты установки, появления или удаления устройства?
- Какие пользовательские файлы, ярлыки и списки недавних объектов указывают на работу со съёмным томом?
- Есть ли признаки запуска утилит очистки или очистки журналов?
- Какие источники прочитаны полностью, частично или недоступны?

Чем приложение не является

- Это не блокиратор записи (write blocker), не средство создания криминалистического образа (forensic image) и не замена EnCase, X-Ways, Magnet AXIOM или Autopsy.
- Оно не выполняет полноценное восстановление удалённых ячеек реестра (carving), MFT/USN-анализ или повторное применение журналов LOG1/LOG2 (replay).
- Оно не доказывает сам факт копирования файла на USB только по LNK, Jump List или букве диска.
- Оно не гарантирует восстановление событий, если Windows их не писала, перезаписала или источник был уничтожен.
- Оно не блокирует устройства и не меняет политики доступа.

Основа методики: [Magnet Forensics — USB Devices Artifact Profile · artefacts.help](#) — [Windows ETW USB activity · DFIR Review](#) — [Partition/Diagnostic 1006 и VSN](#)

2. Архитектура простыми словами

Приложение построено по слоистой схеме, близкой к Clean Architecture. Это нужно не ради терминологии, а чтобы отделить **правила анализа** от **доступа к Windows** и от **интерфейса пользователя**.

Слой	Роль	Что это даёт инженеру
Presentation WPF + MVVM	Окна, вкладки, таблицы, кнопки, статусы, сортировка и экспорт.	Интерфейс не решает, что считать доказательством; он показывает уже рассчитанный результат и пояснения.
Application	Оркестратор сканирования, граф идентичности, сопоставление, временная шкала, поиск очистки.	В одном месте задаются порядок анализа и правила, которые не зависят от конкретного API Windows.
Domain	Модели устройства, доказательства, выявленные проблемы (findings), покрытие проверки и словари отображения.	Все источники приводятся к общей форме, поэтому их можно одинаково сравнивать и сохранять.
Infrastructure	Чтение реестра, Event Log, WMI, файлов и автономных кустов реестра (offline hive); SQLite, JSONL, PDF/Excel/HTML, Procmon и Win32.	Ошибка одного источника не прерывает весь аудит. Она отдельно отражается в покрытии проверки (coverage).

Интерфейс принимает команду «Полное сканирование» и показывает прогресс.



Оркестратор запускает источники в фиксированном порядке и собирает предупреждения.



Сборщики Windows создают записи устройств и доказательств без окончательных выводов.



Аналитические сервисы классифицируют транспорт, объединяют идентичности, связывают тома и рассчитывают даты.



Хранилище и отчёты сохраняют исходный контекст, происхождение данных (provenance), силу доказательства и сведения о неполноте источников.

Почему используется порт/адаптерный подход

Оркестратор знает, что ему нужен «сборщик доказательств» или «хранилище», но не привязан к конкретному реестру, WMI или SQLite. Благодаря этому новый источник можно добавить отдельно; существующая последовательность анализа остаётся понятной и тестируемой.

Архитектурные материалы: [Microsoft — Dependency injection in .NET](#) · [Microsoft — WPF data binding](#)

3. Полный конвейер сканирования

Полное сканирование идёт последовательно в фоновом задании. Порядок важен: сначала собираются первичные записи, затем исторические данные и сведения о текущем состоянии (live-данные). После этого запускается анализ.

Шаг	Что происходит	Зачем
1	Фиксируется время начала, версия Windows, имя компьютера, пользователь, права администратора и дата установки ОС.	Создать контекст сессии и отделить штатные действия Windows Setup от подозрительной очистки.
2	Реестр PnP: USB, USBSTOR, SCSI, WPD, usbflags и MountedDevices во всех ControlSet.	Получить основной инвентарь, идентификаторы, даты и остаточные следы.

3	SetupAPI: текущий и архивные журналы установки устройств.	Подтвердить установку/инициализацию PnP и сохранить текст секции.
4	Windows Event Log и журналы endpoint-защиты.	Получить события с системным временем и конкретными DeviceInstanceId.
5	Профили пользователей: реестр, LNK, Jump Lists, Shellbags, Recycle Bin.	Понять, с какими томами и путями взаимодействовал пользователь.
6	Автономные кусты реестра (offline hive) активных и неактивных профилей.	Получить данные, недоступные через обычный HKU, не изменяя исходный куст реестра.
7	Prefetch, Amcache, Shimcache, PCA, BAM/DAM и Security 4688.	Найти исполнение утилит и контекст, но не подменять им дату подключения USB.
8	Удаляются дубли пользовательских артефактов.	Один и тот же профиль может быть прочитан через live HKU и offline hive.
9	DeviceMigration, сравнение ControlSet, Windows.old, существующие VSS, transaction logs.	Найти более старые или остаточные записи и показать границы восстановления.
10	Live WMI-снимок добавляет состояние «подключено сейчас».	Отделить исторически известное устройство от физически присутствующего в момент аудита.
11	Классификация транспорта и роли.	Не путать UASP, MTP, USB4, хабы, встроенные и виртуальные устройства.
12	Граф идентичности создаёт основные объединённые записи (canonical) физических устройств.	Свести несколько записей устройства Windows (devnode) для одного устройства и не объединить два одинаковых экземпляра.
13	Связываются MountedDevices, Event 1006, VSN, буквы дисков и пользовательские артефакты.	Построить цепочку «физическое устройство → том → пользовательский след».
14	Временная шкала назначает точные и оценочные даты.	Использовать только разрешённые источники для connection date.
15	Поиск очистки и противоречий.	Показать признаки, но не объявлять умышленную очистку без подтверждения.
16	Расчёт покрытия и сохранение сессии.	Инженер видит не только результат, но и что не удалось проверить.

4. Функциональные блоки: что, зачем и как проверяется

4.1. Права администратора и контекст системы

Что проверяется. Приложение нужно запускать с повышенными правами (elevated). В начале сессии фиксируются сведения о Windows и время установки ОС.

Зачем. Без повышения прав могут быть недоступны Security log, часть PnP-свойств, автономные кусты реестра, чужие профили и Procton. Дата установки нужна для трёхчасового льготного периода (grace period): очистка или пересоздание журналов во время Windows Setup не должны автоматически считаться ручной зачисткой.

Что означает результат. Статус администратора говорит о полноте доступа, но не гарантирует, что все журналы включены политикой или сохранились.

Источники: Microsoft — как работает UAC · Microsoft Sysinternals — Process Monitor

4.2. Инвентаризация PnP-реестра: USB, USBSTOR, SCSI и WPD

Что сканируется. Во всех доступных ControlSet* читаются ветви Enum\USB, Enum\USBSTOR, Enum\SCSI и Enum\SWD\WPDBUSENUM. Извлекаются DeviceInstanceId, VID/PID, серийный номер (serial), manufacturer/product, service, Hardware/Compatible IDs, ContainerID, parent, location и даты.

Зачем. Разные классы устройств проявляются по-разному. Обычная флешка чаще видна в USBSTOR; UASP-накопитель может иметь SCSI-представление; телефон — WPD/MTP; составное устройство создаёт несколько интерфейсов.

Что сопоставляется. Одинаковые записи из разных ControlSet и ветвей не объединяются по одному VID/PID. Для физической идентичности требуются более сильные признаки: точный instance ID, ContainerID, аппаратный serial или топология.

Как понимать результат. Запись в Enum означает, что PnP знает или помнит запись устройства Windows (devnode). Это не обязательно означает, что устройство подключено сейчас.

Источники: Microsoft — Device identification strings · Microsoft — DEVPKEY_Device_InstanceId · Magnet Forensics — USB registry artifacts · Belkasoft — Windows Registry forensics

4.3. PnP DevProperties 0064–0067: даты устройства

Что читается. FILETIME-свойства семейства {83da6326-97a6-4088-9453-a1923f573b29} : FirstInstallDate, InstallDate, LastArrivalDate и LastRemovalDate.

Зачем. Они дают структурированные системные даты без разбора локализованного текста.

Как понимать:

- **FirstInstallDate** — когда Windows впервые установила данный экземпляр устройства; обновление драйвера её не меняет.
- **InstallDate** — последняя установка/обновление драйвера; это не обязательно новое физическое подключение.
- **LastArrivalDate / LastRemovalDate** — последнее появление/удаление devnode, если Windows сохранила свойства.

Не путать: «первая установка экземпляра» и «самое первое физическое подключение устройства в его жизни» — разные утверждения. Свойства относятся к данной установке Windows и devnode.

Ограничение текущей версии интерфейса. Приложение сохраняет PnP-дату и учитывает её как точную в Coverage, но форматтер поля «Когда подключали» сейчас не обрабатывает тип PnpDevProperty . Поэтому GUI и отчёт могут показать «точное время неизвестно», хотя значение FirstConnectedUtc внутри сессии заполнено. До исправления интерфейса проверяйте raw device record/SQLite и Coverage, а отсутствие видимой даты не трактуйте как отсутствие PnP-свойства.

Источники: Microsoft — FirstInstallDate · Microsoft — InstallDate · Microsoft — Get-PnpDeviceProperty · NirSoft USBDeview — описание свойств 0064–0067 и оговорки

4.4. ContainerID, parent и LocationPaths

Что проверяется. ContainerID группирует несколько devnode одного физического устройства. ParentId и LocationPaths показывают положение в PnP-дереве и помогают связать USB-часть с SCSI/UASP-представлением.

Зачем. Телефон, док-станция или составное устройство (composite) могут создать несколько интерфейсов MI_xx . Без группировки инженер увидит дубли и может зависеть число устройств.

Ограничение. ContainerID зависит от корректной реализации прошивки и эвристик Windows. LocationPaths может отсутствовать или меняться при подключении к другому порту.

Источники: Microsoft — Overview of Container IDs · Microsoft — USB ContainerIDs · Microsoft — Device Parent · Microsoft — LocationPaths

4.5. usbflags как остаточный продуктовый след

Что сканируется. `ControlSet*\Control\usbflags`. Имена ключей разбираются как возможные VID/PID/revision и обогащаются встроенной базой USB Vendor IDs.

Зачем. Запись может сохраниться, когда основная PnP-ветвь уже отсутствует, и указать на модель/семейство устройства.

Как понимать результат. Это **слабый след на уровне модели продукта (product-level trace)**. VID/PID описывает тип продукта, а не уникальный экземпляр. Запись присоединяется к физическому устройству, только если в текущем наборе есть ровно одна однозначная группа-кандидат.

Источники: USB ID Repository — usb.ids · usbids.us/bids — репозиторий · Microsoft USBView

4.6. MountedDevices: буквы, Volume GUID и подпись диска

Что сканируется. `HKLM\SYSTEM\MountedDevices`. Бинарные значения расшифровываются как буква диска (drive letter), Volume GUID, MBR signature + partition offset, GPT GUID или путь устройства.

Зачем. Эта ветвь связывает логическое имя Windows — например, `E:` — с идентичностью тома/диска. Она служит мостом от устройства к LNK и Jump Lists.

Правило безопасности. Одна буква диска не является доказательством: Windows переиспользует буквы. Привязка к физическому устройству строится по точному PnP instance ID, аппаратному serial, DiskID/signature или подтверждённой WMI-ассоциации.

Источники: Magnet Forensics — MountedDevices и USBSTOR · Vortex Forensic Repository — USB Registry Tracks · DFIR Review — VSN correlation

4.7. SetupAPI.dev.log и архивы

Что сканируется. Текущий `%SystemRoot%\INF\setupapi.dev.log`, сменяемые журналы `setupapi.*.log` и доступные исторические копии. Парсер выделяет разделы с отметкой времени (timestamp) и USB/PnP-идентификаторами, сохраняя исходный текст и номер раздела.

Зачем. SetupAPI документирует установку устройств и драйверов и полезен для подтверждения первого знакомства Windows с экземпляром.

Что означает дата. Это дата операции установки/инициализации, а не универсальный журнал каждого последующего plug/unplug. Удаление устройства или обслуживание драйвера тоже может создавать секции.

Источники: [Microsoft — SetupAPI Text Logs](#) · [Microsoft — Device Installation Log Entries](#) · [Magnet Forensics — forensic use of SetupAPI](#)

4.8. Windows Event Logs

Какие каналы читаются. System, Security, Kernel-PnP Configuration/Device Configuration, Storage-ClassPnP, Partition/Diagnostic, WPD-MTPClassDriver, DeviceSetupManager и DriverFrameworks-UserMode.

Как фильтруется. Запись должна содержать признак устройства (device marker) USB/WPD/UASP/USB4. XML разбирается по структуре: provider, channel, Event ID, Record ID, timestamp, computer и именованные поля. Локализованное сообщение служит дополнительным текстом, а не единственным источником.

Какие события могут задавать дату. Используется явный список разрешённых событий (allowlist): например, Security 6416, Partition 1006, Kernel-PnP 400/410 и определённые события Storage-ClassPnP, если запись идентифицирует конкретное устройство.

Какие не задают дату. Kernel-PnP 411 (ошибка запуска), 420 (удаление конфигурации), 430 (нужна дополнительная установка) и общие WPD/driver events остаются подтверждающими. Они не объявляются физическим отключением.

Важная оговорка: Event 6416 означает распознавание внешнего PnP-устройства, а не обязательно USB-накопителя. Встречаются принтеры и программные устройства; поэтому нужна проверка Device ID и класса.

Источники: [Microsoft — Event 6416](#) · [Microsoft — Audit PNP Activity](#) · [artefacts.help — Kernel-PnP, Storage-ClassPnP, Partition](#) · [Vortex — USB Event Log Traces](#)

4.9. Partition/Diagnostic Event 1006 и VSN

Что извлекается. Идентификаторы устройства, DiskId/signature, geometry/partition data и найденные Volume Serial Numbers.

Зачем. VSN из системного события можно сопоставить с VSN внутри LNK или Jump List. Это позволяет связать пользовательский файл-артефакт с конкретным томом даже после смены буквы.

Как понимать результат. VSN — идентификатор файловой системы тома, а не аппаратный серийный номер накопителя. Форматирование может изменить VSN. У одного устройства может быть несколько томов и несколько VSN.

Источники: [DFIR Review — Recover more VSNs with Event 1006](#) · [artefacts.help — Partition/Diagnostic](#)

4.10. Endpoint Protection / DLP

Что проверяется. Если обнаружена поддерживаемая система защиты конечных устройств (endpoint-среда), приложение читает относящиеся к проверке сообщения Application log и классифицирует их как connect, disconnect, denied или generic.

Зачем. Корпоративный агент может видеть подключение или блокировку там, где стандартный журнал неполон.

Ограничение. Текст и семантика зависят от продукта и версии. «Удалено из политики» не считается физическим отключением; denied/blocked обрабатывается раньше общих слов удаления. Результат всегда нужно сверять с документацией конкретного DLP/EDR.

Источники: [Microsoft — Windows Event Logging](#) · [Microsoft — Querying for Events](#)

4.11. Пользовательские профили и MountPoints2

Что сканируется. Загруженные HKU и автономные NTUSER.DAT/UsrClass.dat не более чем для 256 профилей: MountPoints2, RecentDocs/MRU, ComDlg32, TypedPaths и связанные структуры оболочки (shell). SID сопоставляется с ProfileList и понятным именем профиля.

Зачем. Системные артефакты отвечают «какое устройство знала Windows», а профиль пользователя — «какой пользователь видел том или путь».

Как понимать результат. MountPoints2 связывает профиль с пространством имён тома или удалённого ресурса (volume/remote namespace). Это подтверждает пользовательский контекст, но само по себе не доказывает чтение или копирование конкретного файла.

Источники: [Magnet Forensics — MountPoints2](#) · [Vortex — user attribution through MountPoints2](#)

4.12. LNK, PIDL и Jump Lists

Что сканируется. Ярлыки в Recent, Desktop, Start Menu и SendTo; AutomaticDestinations и CustomDestinations. Структуры читаются с контролем длины; извлекаются target path, volume information, VSN и PIDL-фрагменты.

Зачем. Ярлык или встроенный LNK может сохранять путь и информацию о томе после отключения носителя или удаления исходного файла.

Как сопоставляется. Если в артефакте есть VSN и он точно совпадает с томом устройства, создаётся подтверждающая связь. Одна буква диска намеренно недостаточна.

Как понимать результат. LNK/Jump List показывает, что Windows Shell или приложение создали ссылку или запись на объект. Это не доказывает направление копирования и полный объём действий.

Источники: [Microsoft \[MS-SHLLINK\]](#) · [Microsoft — Shell Links](#) · [Eric Zimmerman — LECmd](#) · [Eric Zimmerman — JLECmd](#)

4.13. Shellbags

Что проверяется. BagMRU-структуры в UsrClass.dat/HKU Classes. Приложение оставляет только узлы с USB/removable/volume marker и извлекает читаемые PIDL-фрагменты.

Зачем. Shellbags могут показать, что пользователь навигировал по папке, даже если папка уже отсутствует.

Как понимать результат. Наличие shellbag подтверждает след навигации. LastWrite конкретного BagMRU нельзя автоматически считать точным временем открытия папки. Его нужно подтверждать LNK, Jump List и другими источниками.

Источники: [Eric Zimmerman — ShellBags Explorer / SBECmd](#) · [Registry Parser — Shellbags caveats](#)

4.14. Recycle Bin metadata

Что сканируется. Только метаданные `$I*` в `$Recycle.Bin\SID` на доступных дисках: исходный путь, размер и время удаления, если структура валидна.

Зачем. Путь может указывать на букву или том и дать контекст удаления файла.

Ограничение. Приложение не заявляет восстановление содержимого `$R` и не доказывает, что файл находился на конкретном физическом USB без дополнительного сопоставления тома.

Источники: [Eric Zimmerman — RBCmd](#) · [RBCmd repository](#)

4.15. Prefetch

Что сканируется. Файлы `%SystemRoot%\Prefetch\*.pf`, связанные с известными утилитами очистки или removable path indicators.

Зачем. Prefetch может подтвердить запуск исполняемого файла и сохранить имя/путь после удаления программы.

Интерпретация. Для известной cleaner-утилиты факт её запуска — прямое доказательство запуска, но не очистки. Обычный removable path имеет косвенную силу. Prefetch никогда не устанавливает дату подключения USB.

Ограничение текущей автоматической аналитики. Prefetch-сборщик помечает cleaner как `CLEANER_EXECUTION`, а CleanupDetector в текущей версии ищет `CLEANER_HINT`. Поэтому такая Prefetch-запись видна в журнале доказательств, но автоматически не превращается в finding «запуск/вероятная очистка». Инженер должен проверить её вручную вместе с 4688, Event 104/1102 и противоречиями источников.

Источники: Eric Zimmerman — [PECmd · Windows Forensic Artifacts knowledge base](#)

4.16. Amcache и Shimcache

Что сканируется. Копия `Amcache.hve` и AppCompatCache из SYSTEM. Ищутся пути/метаданные исполняемых файлов, релевантные USB или cleaner-инструментам.

Зачем. Эти источники помогают подтвердить присутствие или совместимость исполняемого файла и расширяют историю исполнения.

Как понимать результат. Особенно для Shimcache наличие записи не означает доказанный запуск. Поэтому записи маркируются Indirect и не участвуют во временной шкале подключений (connection timeline).

Источники: Eric Zimmerman — [AmcacheParser](#) · Eric Zimmerman — [AppCompatCacheParser](#) · Eric Zimmerman Tools — назначение парсеров

4.17. PCA и BAM/DAM

Что сканируется. `C:\Windows\AppCompat\PCA\PcaAppLaunchDic.txt`, PcaGeneralDb0/1 и registry-ветви BAM/DAM по SID.

Зачем. PCA и BAM/DAM дают дополнительные отметки времени запуска программ и путь к исполняемому файлу (executable). Это помогает установить, с какой утилитой очистки связано событие.

Как понимать результат. PcaAppLaunchDic и BAM/DAM дают подтверждающие сведения о запуске (corroborating execution evidence); общие записи базы PCA (database records) считаются косвенными (Indirect). Доступность зависит от версии Windows, политики и жизненного цикла записи.

Источники: [Velociraptor — AppCompat PCA artifact](#) · [Velociraptor — BAM artifact](#) · [Windows Forensic Artifacts — BAM/DAM caveats](#)

4.18. Security Event 4688: атрибуция процесса

Что проверяется. Security log на Event 4688 для путей известных cleaner/USB-утилит.

Зачем. Если аудит Process Creation был включён, событие содержит исполняемый файл (executable) и учётную запись создателя процесса. Это сильнее, чем косвенное наличие артефакта (artifact presence).

Ограничение. Событие отсутствует, если политика не была включена или журнал перезаписан. Командная строка появляется только при отдельной политике и может содержать чувствительные данные.

Источники: [Microsoft — Event 4688](#) · [Microsoft — Command line process auditing](#)

4.19. Исторические источники: DeviceMigration и ControlSet

Что сканируется. DeviceMigration, различия между ControlSet и текущим выбором `SYSTEM\Select`.

Зачем. После обновления Windows или штатной PnP-очистки старые устройства могут исчезнуть из текущего Enum, но сохраниться в migration-ветвях или другом ControlSet.

Как понимать результат. Это исторический остаточный след (historical residual). LastPresentDate может дать полезную отметку времени, но различие ControlSet само по себе не означает антифорензику. Возможны LastKnownGood, обновление (upgrade) или обычная смена записей.

Источники: [Hacking Exposed CF — PnP cleanup and DeviceMigration](#) · [Ilya Kobzar — Windows updates and USB artifacts](#) · [DFIR Handbook — DeviceMigration](#)

4.20. Windows.old, offline hive и VSS

Что сканируется. Известные кусты SYSTEM/SOFTWARE и журналы SetupAPI в Windows.old, а также до восьми уже существующих теневых копий (shadow copies). Исходный куст сначала копируется и хешируется. Только после этого копия временно подключается для чтения.

Зачем. Предыдущая Windows или снимок (snapshot) может хранить записи, которых уже нет в работающей системе.

Безопасность. Приложение не создаёт новый снимок VSS и не загружает исходный куст напрямую. Журналы транзакций (transaction logs) копируются и хешируются, но подтверждённое повторное применение журналов или восстановление удалённых ячеек (replay/deleted-cell carving) не выполняется.

Источники: [Microsoft — Volume Shadow Copy Service](#) · [Microsoft — Win32_ShadowCopy](#) · [Microsoft — Registry backup under VSS](#)

4.21. Live-снимок и мониторинг

Что проверяется сейчас. WMI-классы Win32_PnPEntity, Win32_DiskDrive и Win32_LogicalDisk; PnP metadata читается для USB, USBSTOR, SCSI/UASP, WPD и USB4 candidates. Буква диска связывается через WMI associations.

Как работает мониторинг. Win32_DeviceChangeEvent типов 2/3 и оконные PnP-уведомления запускают обновление с задержкой устранения дребезга (debounce) 750 мс. Это обновление по событию (event-driven refresh), а не постоянное полное сканирование.

Ограничение fallback. Если WMI watcher не запускается, код выставляет флаг `UsesPollingFallback`, но отдельный периодический polling-цикл в текущей версии не реализован. Остаются оконные PnP-уведомления, если они доступны.

Интерпретация. «Подключено сейчас» подтверждает присутствие во время запроса. «Время подключения» в live-окне — first seen текущей сессией мониторинга, а не историческое время Windows.

Источники: [Microsoft — Win32_PnPEntity](#) · [Microsoft — Win32_DeviceChangeEvent](#) · [Microsoft — PresentOnly semantics](#)

4.22. Классификация UASP, MTP/PTP, USB4 и роли устройства

Что анализируется. Service, instance ID, hardware/compatible IDs, LocationPaths и имя. Определяются транспорт (transport), подключение (connection) и роль/классификация (role/classification).

- **MSC/USBSTOR** — классический USB mass storage.
- **UASP/SCSI** — `uaspstor` или SCSI-узел с подтверждённым USB-bridge/topology.
- **MTP/PTP/WPD** — телефоны/камеры и portable devices, которые не обязаны иметь drive letter.
- **USB4/Thunderbolt** — явные router/service/ID markers; PCIe-узел без достаточного подтверждения остаётся candidate.
- **Hub, BuiltIn, Composite, Virtual, External** — роль, чтобы инфраструктура шины не выглядела как пользовательская флешка.

Принцип. Если признаков недостаточно, выбирается Unknown, а не догадка по названию продукта.

Источники: Microsoft — USB class drivers, Usbstor/Uaspstor · Microsoft — WPD architecture · Microsoft — USB4 Connection Manager · Microsoft — UCSI / USB-C

4.23. Сторонние утилиты и Procmon-трассировка

Что делает блок. Находит запущенные USBDetector, USBDeview или USB Oblivion. Считывает видимые таблицы через Win32/UI Automation или резервное чтение буфера обмена (clipboard fallback), приводит столбцы к общей форме и позволяет выполнить строгую трассировку операций с реестром через Procmon.

Зачем. Это независимое сравнение с внешним инструментом и способ понять, из каких ключей конкретная строка могла быть получена.

Интерпретация. Захваченная строка — вывод сторонней программы, а не первичный артефакт UsbForensicAudit. Procmon показывает обращения процесса после старта трассировки и не восстанавливает прошлые обращения.

Источники: NirSoft — USBDeview · Microsoft Sysinternals — Process Monitor · Microsoft — UI Automation

5. Как приложение сопоставляет записи

5.1. Граф физической идентичности

Windows часто создаёт несколько записей для одного корпуса: USB parent, USBSTOR disk, SCSI/UASP disk, WPD interface, volume и несколько `MI_xx`. Приложение строит группы по сильным ключам:

1. точный нормализованный DeviceInstanceId;
2. валидный ContainerID;
3. аппаратный serial, если он не похож на сгенерированный Windows instance string;
4. совпадающая топология: ParentIdPrefix или LocationPaths;
5. composite parent после удаления интерфейсной части `MI_xx`.

По самому сильному доступному ключу вычисляется стабильный идентификатор основной объединённой записи (canonical ID) вида `DEV-...`. Он нужен для внутреннего объединения и не является заводским идентификатором устройства.

Защита от ложного объединения (merge). Два устройства одной модели имеют одинаковые VID/PID. Поэтому VID/PID не используются как самостоятельный ключ физической идентичности и не переносят даты между serial A и serial B.

5.2. Связь «устройство → том»

Сопоставления MountedDevices (mappings) сначала объединяются по двоичному отпечатку (binary fingerprint). Затем сопоставление присоединяется к физическому устройству только при наличии сильного основания: точного пути PnP (exact PnP path), аппаратного serial в device path или подтверждённой текущей связи WMI (live WMI association).

5.3. Связь «том → пользовательский артефакт»

LNK/Jump List/PIDL должен содержать VSN. Если он совпадает с VSN уже привязанного тома, создаётся производная подтверждающая запись сопоставления (derived correlation evidence). Совпадение VSN и буквы диска даёт уверенность High; один VSN — Medium. Одна буква диска не используется.

5.4. Общее сопоставление доказательств

Для каждой основной объединённой записи устройства выбираются до 30 относящихся к ней записей по сильным признакам: instance ID, serial, ContainerID, ParentId и linked source IDs. Производные записи маркируются **Corroborating**, содержат происхождение данных (provenance) и не могут назначать дату подключения.

Источники принципов: Microsoft — ContainerID grouping · Microsoft — Device Instance IDs · DFIR Review — VSN attribution to LNK/Jump Lists

6. Как читать результаты

6.1. EvidenceStrength

Уровень	Практический смысл	Пример
Direct	Источник прямо фиксирует описанное действие или системное событие.	Event 4688 прямо фиксирует запуск процесса; PnP allowlist event прямо фиксирует конфигурацию/появление конкретного instance ID.
Corroborating	Подтверждает цепочку, но не является достаточным в одиночку.	VSN-совпадение LNK с томом, BAM/PCA, событие драйвера без разрешения задавать connection date.
Indirect	Косвенный или остаточный след; требует независимого подтверждения.	Shimcache presence, общий MRU, исторический residual.

Важно: Direct относится к утверждению записи. Например, Prefetch может прямо доказать запуск cleaner-программы, но не доказать, что программа действительно очистила USB-следы.

6.2. Confidence

Уровень уверенности (Confidence) High/Medium/Low характеризует качество конкретного сопоставления. Это не математическая вероятность. В отчёте всегда нужно читать сведения о происхождении данных (provenance): почему выставлен этот уровень и какие ключи совпали.

6.3. Типы дат

Отображение	Что означает	Что проверить
PnpDevProperty	Дата взята из структурированного свойства PnP.	Какое именно свойство: FirstInstall, LastArrival или LastRemoval; учесть текущий дефект отображения FirstConnectedText.
ExactEvent	Дата взята из разрешённого события/SetupAPI, идентифицирующего устройство.	Provider, Event ID, Record ID, DeviceHint и raw XML/text.
LiveAtScan	Устройство впервые увидено приложением в текущей live-сессии.	Не выдавать за историческую дату подключения.
LastActivityEstimate	Точного отключения нет; показана последняя активность как оценка.	Искать LastRemoval или disconnect event.
Unknown	Windows помнит устройство, но достоверная дата отсутствует.	Проверить coverage и исторические источники.

6.4. Категории строк устройства

- **RealUsb** — полноценный USB/Type-C device record.
- **RelatedStorage** — disk/volume record, связанный с USB, но не отдельный физический корпус.
- **UsbFlagsTrace** — слабый остаточный VID/PID trace.
- **SupportArtifact** — инфраструктурная запись Windows; её нельзя считать пользовательским носителем.

7. Как ищутся признаки очистки

CleanupDetector не «угадывает виновного». Он формирует выявленные проблемы (findings) из наблюдаемых сигналов и явно показывает assessment, severity, initiator, possible tool, confidence и provenance.

Сигнал	Почему интересен	Почему не является автоматическим приговором
Security Event 1102 / System Event 104	Журнал был очищен.	Очистка может быть административной, установочной или штатной; важны account и время.
setupapi.dev.log отсутствует, мал или создан значительно позже Windows	Возможна ротация/пересоздание.	Обновление, восстановление ОС или обслуживание также меняют файл.
USBSTOR есть, а SetupAPI-секции отсутствуют	Источники противоречат друг другу.	SetupAPI мог ротироваться; Windows.old/DeviceMigration объясняют разницу.
MountedDevices есть, а Enum USB отсутствует	Остался volume mapping без device record.	Это может быть нормальный stale mapping или миграция.
Запуск USB Oblivion/cleaner в 4688 или execution artifacts	Инструмент способен удалять следы.	Сам запуск не доказывает использование функции очистки; Prefetch cleaner сейчас требует ручной проверки из-за несовпадения внутренних меток.
Cleaner рядом по времени с 104/1102 или registry contradiction	Несколько независимых сигналов могут образовать вероятную цепочку.	Автоматическая ветка Prefetch cleaner сейчас не срабатывает; статус Probable допустим только после фактического сопоставления доступных записей.

События в первые три часа после установки Windows маркируются как **OsInstall / Informational**, если совпадают с контекстом Windows Setup. Это снижает ложные обвинения.

Источники: Microsoft — Event 1102 · MITRE ATT&CK — Clear Windows Event Logs · Microsoft — Procmon for registry/process correlation

8. Практический порядок работы инженера

8.1. Перед запуском

1. Зафиксировать цель: инвентаризация, реагирование на инцидент (incident response), проверка конкретного serial или поиск очистки.
2. Записать локальное время, часовой пояс и состояние ПК. Приложение хранит UTC и показывает МСК; для другой зоны учитывать преобразование.
3. Не подключать новые устройства без необходимости: это создаёт новые PnP и SetupAPI-следы.
4. Запустить от имени администратора. Если исследование юридически значимо, сначала создать forensic-образ штатным процессом организации.

8.2. После полного сканирования

1. **Сначала Coverage.** Если EventLog, SetupAPI или профили Partial/Error, нельзя делать вывод «следов нет».
2. **Затем USB Devices.** Отфильтровать RealUsb, проверить canonical ID, serial, ContainerID, transport и происхождение классификации (classification provenance).
3. **Проверить даты.** Различать PnpDevProperty, ExactEvent и оценку (estimate); открыть связанные доказательства (evidence).
4. **Проверить тома.** Сверить VSN, Volume GUID, DiskId и drive letter. Не использовать одну букву как идентификатор.
5. **Проверить пользователя.** Искать MountPoints2, LNK, Jump List и Shellbags с совпадающим VSN/путём.
6. **Проверить Cleanup Findings.** Начинать с High/Probable, затем читать происхождение данных (provenance) и исходные доказательства (raw evidence).
7. **Сформировать полный отчёт.** Сводный отчёт удобен руководителю, но не заменяет журнал доказательств.

8.3. Как расследовать конкретную флешку

1. Найти аппаратный serial и DeviceInstanceId в USBSTOR/USB.
2. Проверить ContainerID и linked source IDs — нет ли UASP/SCSI или composite-узлов.
3. Открыть evidence с точным serial/instance ID.
4. Проверить FirstInstall/LastArrival/LastRemoval и SetupAPI.
5. Найти volumes: VSN, DiskId, drive letter.
6. Перейти к LNK/Jump Lists/MountPoints2 с тем же VSN.
7. Сопоставить с пользователем и временем, учитывая силу доказательства.
8. Если есть cleaner finding — проверить 4688, Prefetch/PCAT/BAM и Event 104/1102 в окне времени.

8.4. Что писать в инженерном заключении

Корректно: «Windows хранит запись устройства USBSTOR с serial X. PnP LastArrivalDate указывает на 10.07.2026 14:03 МСК. Событие Kernel-PnP 410 с тем же DeviceInstanceId подтверждает запуск devnode. LNK пользователя Y содержит VSN тома Z, совпадающий с привязанным томом устройства».

Некорректно: «Пользователь Y скопировал файл на флешку в 14:03», если нет отдельного файлового журнала или иных прямых доказательств копирования.

9. Хранение, отчёты и проверяемость

9.1. Где сохраняются данные

Если рядом с EXE разрешена запись, используется `{exe}\data`. Иначе применяется резервный путь (fallback) `%LOCALAPPDATA%\UsbForensicAudit`. Инженер должен записать фактический путь из интерфейса (UI) или журнала.

9.2. SQLite

`audit.sqlite` хранит сессии, devices, evidence, cleanup findings, coverage и JSON-представление записей. SessionId отделяет один аудит от другого; уникальные record keys защищают от повторной вставки той же записи внутри сессии.

9.3. JSONL hash chain

`evidence.jsonl` — журнал только для добавления записей (append-only). Каждая строка связана с предыдущей через SHA-256. Это позволяет при обычной проверке обнаружить разрыв или изменение цепочки.

Граница гарантии. Цепочка хешей (hash chain) не является цифровой подписью и не защищает от атакующего, который может полностью переписать файл и пересчитать всю цепочку. Для юридической сохранности нужны внешний хеш (hash), защищённое хранилище, журнал передачи и процедура контроля сохранности доказательств (chain of custody).

9.4. Coverage

Для каждого сборщика (collector) показываются Complete, Partial, Error или NotRun, число записей, сработавший лимит и текст ошибки. ExactDateCoveragePercent — доля основных объединённых записей устройств, у которых дата получена из ExactEvent или PnP property.

Нулевой результат при Error/NotRun означает «источник не проверен», а не «артефактов нет».

9.5. Отчёты

- **HTML** — генератор реализован программно и содержит timeline, dossiers, evidence и coverage, но отдельной кнопки HTML в текущем GUI нет.
- **Полный PDF/Excel** — инженерный результат с устройствами, доказательствами, provenance и coverage.
- **Сводный PDF/Excel** — краткая управленческая выборка; не содержит всей глубины первичных записей.

Область отчёта (Report scope) отфильтровывает внутренние SATA/NVMe и RAM. USB hubs, composite и support artifacts могут присутствовать как инфраструктурный контекст, но не должны считаться пользовательскими накопителями.

Источники: SQLite — Database File Format · NIST FIPS 180-4 — SHA-256 · RFC 3227 — Evidence Collection and Archiving Guidelines

10. Ограничения и запрещённые выводы

Наблюдение	Что допустимо утверждать	Что нельзя утверждать без дополнительных данных
USBSTOR/Enum запись	Windows знает или знала этот devnode.	Устройство подключено сейчас; пользователь копировал файлы.
FirstInstallDate	Экземпляр впервые установлен в данной Windows.	Это первое подключение устройства вообще.
LastArrivalDate	Последнее системно записанное появление devnode.	Это гарантированно каждое физическое подключение.
SetupAPI timestamp	В это время шла установка/инициализация PnP.	Это универсальный plug event.
Kernel-PnP 420	Удалена конфигурация устройства.	Пользователь физически выдернул кабель в эту секунду.
Drive letter	Windows связывала букву с mapping.	Буква всегда принадлежала одному устройству.
LNK/Jump List	Shell/приложение создали ссылку на объект/том.	Файл был скопирован именно на USB.
Shellbag	Профиль хранит след навигации по папке.	LastWrite — точное время открытия или копирования.
Prefetch/4688	Программа запускалась.	Она выполнила очистку или конкретную операцию.
Отсутствующий артефакт	Источник не содержит доступной записи.	Устройство никогда не подключалось или след умышленно удалён.

VID/PID совпадает	Совпадает модель/семейство продукта.	Это один и тот же физический экземпляр.
VSS/Windows.old	Найдена историческая копия доступного source.	Восстановлены все удалённые записи.

Факторы, снижающие полноту

- журнал был отключён политикой, ограничен размером или перезаписан;
- обновление Windows выполнило PnP cleanup;
- устройство не имеет аппаратного serial;
- MTP не создаёт drive letter как обычный накопитель;
- UASP проявляется как SCSI, и USB-bridge не сохранился;
- профиль зашифрован, удалён, повреждён или недоступен;
- сработал защитный лимит collector — это отражается в Coverage;
- живое исследование само неизбежно создаёт минимальные системные изменения.

Известные расхождения текущей версии, важные для эксплуатации

- PnP-дата типа `PnpDevProperty` хранится и учитывается Coverage, но не выводится корректно через текущий форматтер `FirstConnectedText`.
- Prefetch cleaner получает метку `CLEANER_EXECUTION`, тогда как автоматический CleanupDetector ожидает `CLEANER_HINT`.
- HTML-генератор существует в сервисе отчётов, но пользовательская вкладка предоставляет только PDF и Excel.
- Флаг WMI polling fallback существует, но периодический polling при отказе watcher не реализован.

11. Каталог проверенных материалов

Ниже перечислены материалы, использованные как техническая основа. Источники Microsoft описывают смысл данных на уровне платформы. DFIR-материалы и репозитории объясняют практическую интерпретацию артефактов. Дата проверки ссылок: 11.07.2026.

Microsoft Learn и открытые спецификации

1. [DEVPKEY_Device_FirstInstallDate](#)
2. [DEVPKEY_Device_InstallDate](#)
3. [DEVPKEY_Device_InstanceId](#)
4. [DEVPKEY_Device_ContainerId](#)
5. [Overview of Container IDs](#)
6. [USB ContainerIDs in Windows](#)
7. [DEVPKEY_Device_LocationPaths](#)
8. [SetupAPI Text Logs](#)
9. [SetupAPI Device Installation Log Entries](#)
10. [Security Event 6416](#)
11. [Security Event 1102](#)
12. [Security Event 4688](#)
13. [USB Device Class Drivers Included in Windows](#)
14. [Windows Portable Devices Architecture](#)
15. [USB4 Connection Manager](#)
16. [UCSI Driver](#)
17. [\[MS-SHLLINK\] Shell Link Binary File Format](#)
18. [Volume Shadow Copy Service](#)
19. [Sysinternals Process Monitor](#)
20. [USBView и исходный код USBView](#)

DFIR-исследования и учебные материалы

1. [Magnet Forensics — Artifact Profile: USB Devices](#)
2. [DFIR Review — Recover more Volume Serial Numbers with Partition/Diagnostic](#)
3. [artefacts.help — Devices and USB activity](#)
4. [Vortex Forensic Repository — USB Registry Tracks](#)
5. [Vortex Forensic Repository — USB Event Log Traces](#)
6. [Hacking Exposed Computer Forensics — Windows PnP cleanup](#)
7. [Ilya Kobzar — Windows updates and USB artifacts](#)
8. [Velociraptor — AppCompat PCA](#)
9. [Velociraptor — BAM](#)
10. [MITRE ATT&CK — Clear Windows Event Logs](#)

Репозитории и эталонные инструменты

1. Eric Zimmerman Tools
2. LECmd — LNK parser
3. JLECmd — Jump List parser
4. AmcacheParser
5. AppCompatCacheParser
6. Windows Forensic Artifacts knowledge base
7. The USB ID Repository / GitHub mirror
8. NirSoft USBDeview

Стандарты хранения и доказательности

1. SQLite Database File Format
2. NIST FIPS 180-4 — Secure Hash Standard
3. RFC 3227 — Guidelines for Evidence Collection and Archiving

Итог для инженера

UsbForensicAudit следует рассматривать как **средство сопоставления и аудита артефактов работающей Windows**. Его сильная сторона — не один «магический ключ», а последовательная цепочка:

PnP-идентичность → системное событие → том → пользовательский артефакт → контекст запуска и очистки (execution/cleanup context) → покрытие проверки и происхождение данных (coverage и provenance).

Самый надёжный вывод получается, когда совпадают независимые сильные признаки: аппаратный serial или точный instance ID, ContainerID/topology, системное событие, volume fingerprint/VSN и пользовательский артефакт. Если совпадает только название, VID/PID или буква диска, вывод должен оставаться ограниченным и вероятностным.